

Distributed System notes 3

Trao đổi thông tin

Key Term	Vietnamese Translation
Protocol	Giao thức
TCP (Transmission Control Protocol)	Giao thức điều khiển truyền vận
UDP (User Datagram Protocol)	Giao thức gói tin người dùng
Connection-oriented	Hướng kết nối
Reliable protocol	Giao thức tin cậy
Unreliable protocol	Giao thức không tin cậy
Connectionless	Không hướng kết nối
Packet	Gói tin
Acknowledgment (ACK)	Xác nhận
Retransmission	Truyền lại
Streaming	Truyền phát
VoIP (Voice over IP)	Thoại qua Internet
HTTP/HTTPS	Giao thức truyền siêu văn bản (bảo mật)
SSL/TLS	Lớp bảo mật tầng socket / lớp bảo mật truyền tải
FTP (File Transfer Protocol)	Giao thức truyền tệp
SMTP (Simple Mail Transfer Protocol)	Giao thức truyền thư đơn giản
ICMP	Giao thức điều khiển thông báo Internet
RPC (Remote Procedure Call)	Gọi thủ tục từ xa
Client	Máy khách
Server	Máy chủ
Stub	Mã trung gian (Stub)
gRPC	Giao thức RPC của Google
JSON-RPC	RPC sử dụng JSON
XML-RPC	RPC sử dụng XML
Message Oriented Middleware	Phần mềm trung gian hướng thông điệp
Message Queue	Hàng đợi thông điệp
Producer	Bộ gửi thông điệp
Consumer	Bộ nhận thông điệp

Key Term	Vietnamese Translation
Delay	Độ trễ
Asynchronous communication	Giao tiếp bất đồng bộ

Review lại kiến thức: TCP, UDP

<https://viblo.asia/p/tim-hieu-giao-thuc-tcp-va-udp-jvEla11xlkw>

1. TRAO ĐỔI THÔNG TIN GIỮA CÁC TIẾN TRÌNH

Giao thức (Protocol): Là một tập hợp các quy tắc và định dạng được sử dụng để truyền tải dữ liệu giữa các thiết bị trong mạng máy tính hoặc giữa các phần mềm. Giao thức xác định cách thức các hệ thống giao tiếp, bao gồm cấu trúc dữ liệu, cách truyền nhận, và các quy định về cách xử lý lỗi, bảo mật, đồng bộ hóa, và các yêu cầu khác trong quá trình truyền thông.

Các loại giao thức phổ biến:

- **Giao thức mạng:** Các giao thức điều phối cách dữ liệu được truyền qua mạng. Ví dụ:
 - **TCP/IP:** Giao thức truyền tải thông tin qua mạng Internet.
 - **HTTP/HTTPS:** Giao thức truyền tải văn bản, dùng trong trình duyệt web.
- **Giao thức bảo mật:** Các giao thức bảo vệ thông tin trong quá trình truyền tải. Ví dụ:
 - **SSL/TLS:** Dùng để mã hóa kết nối giữa các hệ thống trong mạng.
- **Giao thức ứng dụng:** Dùng cho việc trao đổi dữ liệu giữa các ứng dụng. Ví dụ:
 - **FTP:** Giao thức truyền tệp giữa các máy tính.
 - **SMTP:** Giao thức gửi email.

Các giao thức thường có những quy định về:

- Cấu trúc thông điệp
- Kích cỡ thông điệp
- Thứ tự gửi thông điệp
- Cơ chế phát hiện thông điệp hỏng hay bị mất

Các loại giao thức trong mạng máy tính có thể được phân loại dựa trên hai tiêu chí chính: **hướng kết nối (connection-oriented)** và **tin cậy (reliable)**

Giao thức không hướng kết nối (Connectionless)

□ Định nghĩa: Giao thức không yêu cầu thiết lập hoặc duy trì kết nối trước khi gửi dữ liệu. Dữ liệu được gửi đi ngay dưới dạng các gói độc lập.

□ Đặc điểm:

- Dữ liệu có thể bị mất hoặc đến không theo thứ tự.
- Không có cơ chế xác nhận hoặc sửa lỗi.
- Nhanh hơn vì không cần thiết lập kết nối.

□ Ví dụ giao thức:

- UDP (User Datagram Protocol): Giao thức không hướng kết nối phổ biến nhất, được sử dụng cho các ứng dụng yêu cầu tốc độ cao như streaming video, VoIP.

Ví dụ: xem streaming video trên youtube, twitch, tiktok live, dữ liệu truyền tải yêu cầu trực tiếp trong thời gian thực, dữ liệu truyền tải có thể bị giật, lag. Dữ liệu có thể mất

- ICMP (Internet Control Message Protocol): Giao thức không hướng kết nối dùng để truyền thông tin lỗi hoặc trạng thái mạng, ví dụ lệnh ping.

Giao thức tin cậy (Reliable Protocol)

□ Định nghĩa: Giao thức đảm bảo rằng dữ liệu được truyền đi một cách đáng tin cậy, không mất mát, không bị trùng lặp, và đến đích theo đúng thứ tự.

□ Đặc điểm:

- Sử dụng cơ chế xác nhận (acknowledgment) và truyền lại (retransmission) khi phát hiện lỗi.

Muốn thiết lập kết nối giữa hai máy cần xác nhận (ACK)

- Thích hợp cho các ứng dụng yêu cầu độ chính xác cao. (Gửi file giữa hai người dùng)

□ Ví dụ giao thức:

- TCP: Tin cậy, vì nó có cơ chế số thứ tự, xác nhận, và truyền lại dữ liệu bị mất.

- SCTP: Cũng là một giao thức tin cậy, hỗ trợ đa luồng dữ liệu trong một kết nối.

Giao thức không tin cậy (Unreliable Protocol)

□ Định nghĩa: Giao thức không đảm bảo dữ liệu sẽ đến đích hoặc đến theo đúng thứ tự. Các gói dữ liệu có thể bị mất mà không có cơ chế khắc phục.

□ Đặc điểm:

- Đơn giản, nhanh chóng, nhưng không đảm bảo chất lượng.

- Phù hợp với các ứng dụng mà mất một phần dữ liệu không gây ảnh hưởng lớn.

□ Ví dụ giao thức:

- UDP: Không tin cậy, vì nó không có cơ chế xác nhận hoặc truyền lại. (ví dụ streaming video)

- ICMP: Không tin cậy, vì chỉ dùng để trao đổi thông tin về lỗi hoặc kiểm tra trạng thái mạng.

★ Các tiến trình trao đổi thông tin với nhau thông qua chủ yếu 2 giao thức TCP và UDP

Khi nào nên chọn TCP hoặc UDP trong trao đổi thông tin giữa tiến trình?

□ Chọn TCP: Khi cần độ tin cậy, dữ liệu phải đến đúng thứ tự, ví dụ:

- Gửi file, email. Dịch vụ web.
- Ứng dụng ngân hàng. : Cần chính xác 100%

□ Chọn UDP: Khi cần tốc độ và chấp nhận mất mát dữ liệu nhỏ, ví dụ:

- Truyền thông trực tuyến (video, âm thanh): đề cao tốc độ, có thể mất dữ liệu, lag
- Ứng dụng IoT cần nhẹ và nhanh: Các thiết bị IoT cảm biến nhiệt độ, độ ẩm, chuyển động v.v cần thời gian thực và cập nhật liên tục (không lo về vấn đề bảo mật)
- Trò chơi trực tuyến: Thời gian thực cao

Lời gọi thủ tục từ xa (RPC)

Remote Procedure Call

Khi một tiến trình trên client muốn thực hiện một thủ tục nào đó nằm trên server:

- Thực hiện một lời gọi thủ tục từ xa tới server.
- Tiến trình gọi trên client sẽ vào trạng thái chờ (treo), và quá trình thực thi thủ tục được gọi diễn ra trên server dựa trên các tham số được truyền đến từ client và kết quả sẽ được truyền trở lại cho client tương ứng.

Ví dụ:

Khi chúng ta lên web shopee trả tiền bằng thẻ VCB

Chi tiết về quá trình:

1. **Client:** Gửi yêu cầu gọi thủ tục (payment transaction) đến máy chủ. Các tham số như số thẻ, mã OTP, và thông tin giao dịch sẽ được gửi đến máy chủ.
 2. **Server:** Tiến hành xử lý yêu cầu. Máy chủ kiểm tra thông tin thẻ VCB, xác minh và thực hiện giao dịch.
 3. **Client:** Máy khách tiếp tục "chờ" trong trạng thái treo cho đến khi nhận được kết quả từ máy chủ, chẳng hạn như "giao dịch thành công" hoặc "giao dịch thất bại".
- **Tham số client (Shopee):** Trong ví dụ trên, tham số client là thông tin giao dịch như `card_number`, `expiration_date`, và `amount` được gửi từ client (Shopee) tới server (payment server).
 - **Code chạy thanh toán VCB :** Server xử lý yêu cầu thanh toán bằng cách kiểm tra số thẻ, nếu hợp lệ thì trả kết quả "Payment processed successfully!", nếu không thì trả "Invalid card number".

```
CheckPayment(card_number, expiration_date, amount)
```

Cả client và server sẽ trao đổi dữ liệu qua giao thức mạng, chẳng hạn như HTTP hoặc một giao thức RPC cụ thể (thường là gRPC, SOAP, hoặc JSON-RPC).

RPC giúp đơn giản hóa quá trình lập trình, khiến nó dễ dàng hơn khi gọi các dịch vụ từ xa mà không cần phải lo lắng về chi tiết truyền thông hay mạng.

Đặc điểm của RPC

- Minh bạch: RPC giúp lập trình viên gọi thủ tục từ xa mà không cần quan tâm đến chi tiết truyền thông mạng. Nó giống như gọi một hàm cục bộ => Cơ chế truy cập trong suốt với người dùng
- Đồng bộ hóa: Gọi RPC thường là đồng bộ, tức là chương trình máy khách sẽ chờ kết quả từ máy chủ.
- Tầng giao thức: RPC thường sử dụng các giao thức mạng như TCP hoặc UDP để truyền tải dữ liệu.

Thành phần của RPC

- Client Stub: Là mã chương trình phía máy khách, chịu trách nhiệm đóng gói tham số và gửi yêu cầu đến máy chủ. (Tham số trong thẻ ngân hàng, thông tin người dùng)
- Server Stub: Là mã chương trình phía máy chủ, nhận yêu cầu từ Client Stub, giải mã tham số, thực hiện thủ tục, và gửi trả kết quả. (lấy tham số và thực hiện yêu cầu thanh toán)
- Protocol: Là giao thức truyền thông giữa Client Stub và Server Stub (thường là TCP/IP).

Biến thể và công nghệ liên quan

- gRPC: Một framework RPC hiện đại từ Google, sử dụng giao thức HTTP/2 để truyền dữ liệu và Protobuf để mã hóa.
- XML-RPC: RPC sử dụng XML để mã hóa dữ liệu và HTTP để truyền dữ liệu.
- JSON-RPC: RPC sử dụng JSON làm định dạng trao đổi dữ liệu.

Tính mở của RPC

- Client và Server được cài đặt bởi các NSX khác nhau (không cần quan tâm Hệ Điều Hành có tương thích)
- Giao diện thống nhất client và server (có một giao thức chung để trao đổi thông tin)

Không phụ thuộc công cụ và ngôn ngữ lập trình (Chỉ cần định nghĩa đúng giao thức khi sử dụng framework để truyền thông tin hoặc nhận thông tin, không quan trọng sử dụng ngôn ngữ gì)

Mô tả đầy đủ và trung lập

Thường dùng ngôn ngữ định nghĩa giao diện (Có những trường thông tin nào, thứ tự xuất hiện như nào, xử lý như nào)

Trao đổi thông tin hướng thông điệp

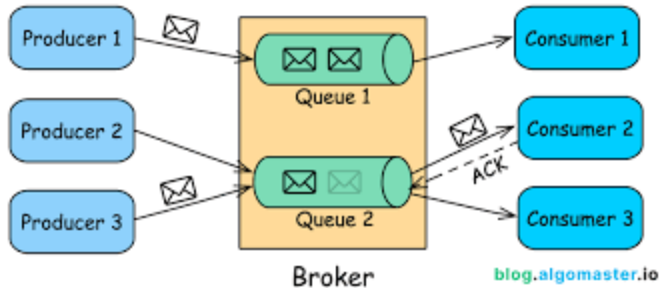
Trao đổi thông tin hướng thông điệp tạm thời (TCP)

Trao đổi thông tin hướng thông điệp bền vững

1. **Message Oriented Middleware:** Hệ thống hàng đợi thông điệp không đồng bộ của hệ thống email, có thể chấp nhận delay và lưu trữ vào hàng đợi thông tin trung gian (nếu hệ thống có trục trặc, đường truyền, v.v), tuy nhiên, messages được lưu bền vững

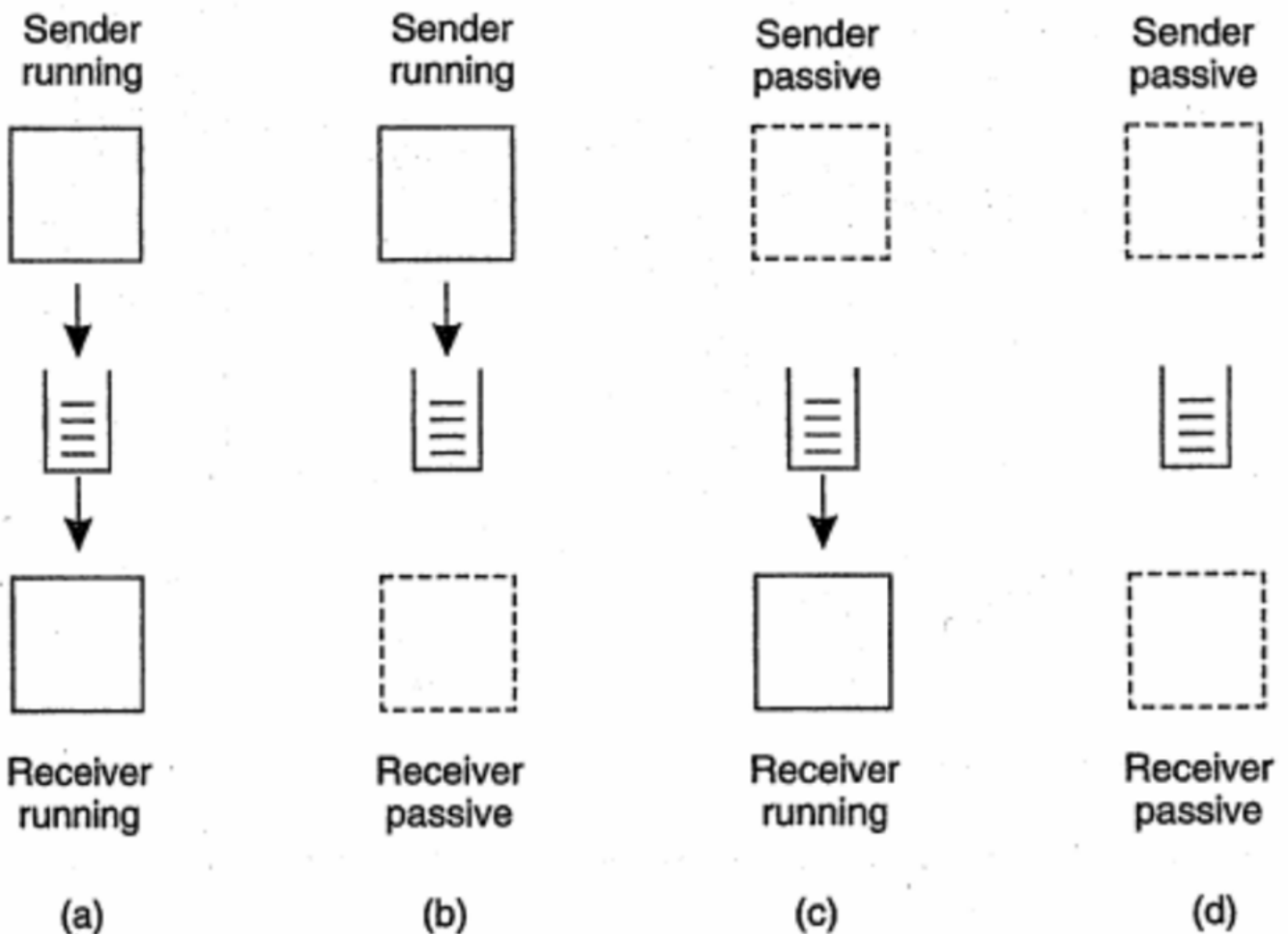
2. Message Queues:

1. Tin nhắn được đưa vào hàng đợi và gửi đến một hoặc nhiều địa chỉ cố định, xử lý theo thứ tự (tin nhắn đến trước, xử lý trước)
2. phổ biến RabbitMQ, Apache ActiveMQ
3. Ví dụ trong hình, PProducer 1 gửi tin nhắn đến hàng đợi và hàng đợi gửi đến Consumer 1. Produce 2,3 gửi vào hàng đợi 2 và gửi đến consumer 2,3.
 1. Số người nhận tin nhắn và số người gửi tin nhắn có thể khác nhau và nhiều hơn



4. Thông thường tin nhắn sẽ được định nghĩa sẵn gửi đi đâu
 1. ví dụ, Producer 3 nhắn cho consumer 2,3

Mô hình



Có 4 mô hình

Sender Running (Người gửi đang hoạt động):

Người gửi đang chủ động gửi tin nhắn hoặc dữ liệu vào hệ thống. Điều này có thể là khi người gửi (producer) đang truyền tải thông tin hoặc dữ liệu đến người nhận (consumer) hoặc hệ thống mà người nhận đang chờ nhận tin nhắn.

Người gửi đang hoạt động nghĩa là nó đang thực hiện nhiệm vụ gửi tin nhắn hoặc tạo ra dữ liệu.

Sender Passive (Người gửi bị động):

Người gửi không chủ động gửi tin nhắn. Thay vào đó, người gửi chỉ chuẩn bị hoặc đang chờ đợi để có thể gửi tin nhắn. Đây có thể là khi người gửi không phải là bên chủ động trong việc truyền tải dữ liệu, có thể chỉ gửi khi có yêu cầu hoặc khi có một sự kiện kích hoạt.

Người gửi không gửi bất kỳ tin nhắn nào trừ khi có một kích hoạt từ phía người nhận hoặc hệ thống.

Receiver Running (Người nhận đang hoạt động):

Người nhận đang chủ động nhận và xử lý tin nhắn từ người gửi. Trong trường hợp này, người nhận không chỉ chờ đợi mà còn thực hiện hành động xử lý tin nhắn hoặc dữ liệu mà người gửi đã gửi.

Người nhận đang chờ và sẵn sàng xử lý hoặc tiêu thụ dữ liệu ngay lập tức khi có tin nhắn đến.

Receiver Passive (Người nhận bị động):

Người nhận không chủ động nhận tin nhắn. Nó chỉ có thể nhận tin nhắn khi có sự kiện hoặc yêu cầu cụ thể từ phía người gửi. Nếu không có tín hiệu hoặc tin nhắn đến, người nhận không làm gì cả.

Người nhận không chủ động trong việc thu thập dữ liệu và chỉ nhận khi có sự kiện xảy ra (chẳng hạn như người gửi gửi tin nhắn).

3. Pubsub Model (publisher - subscriber)

Khái niệm: cho phép các thành phần gửi tin nhắn (publishers) và nhận tin nhắn (subscribers) mà không cần biết đến nhau.

Message broker là công cụ thực hiện mô hình.

Chức năng:

Publishers phát tán các sự kiện hoặc tin nhắn đến chủ đề (topics).

Subscribers đăng ký nhận thông báo từ các chủ đề mà họ quan tâm

Kafka là một platform phổ biến của pub sub model để truyền tải thông tin

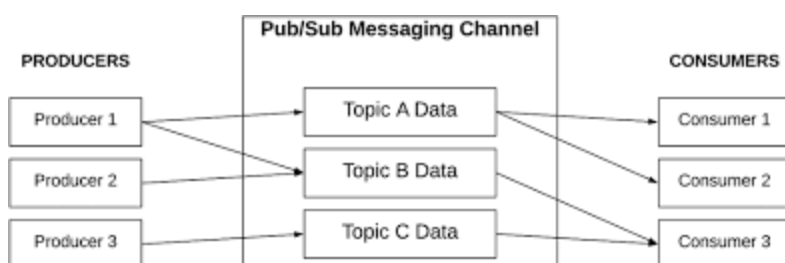
Ví dụ:

Chẳng hạn như Publisher 1 gửi thông tin với topic: payment, thì subscribers (người nhận) nào đăng ký nhận thông tin từ topic payment sẽ nhận được tin nhắn

=> Không giống với message queues phải định nghĩa người nhận, người nhận subscribe topic nào sẽ nhận được tin nhắn từ topic đó.

Có thể subscribe nhiều topic và publish nhiều topic cùng một lúc,

* Có thể unsubscribe



Bài tập

Làm bài tập trông phần slides buổi 3, nộp trên blog
screenshot lại các phần quan trọng